

UNIVERSITY SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY
GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY
Sector 16C, Dwarka, Delhi 110075, India



**MAJOR PROJECT - PROGRESS REPORT
(ICT - 456)**

RigCraft — AI-Powered PC Build Recommendation Engine

**Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology in Computer Science Engineering**

**Submitted By:
Pratik Mehata
(41216403222)
B. Tech CSE (8th Sem)**

**Under the Guidance of
Prof. R.K Purwar**

**Submitted to
University School of Information, Communication and Technology
Guru Gobind Singh Indraprastha University,
Dwarka, New Delhi**

DECLARATION

The features outlined in this progress report have been successfully implemented, tested, and integrated into the final software application, concluding the practical implementation phase of this Major Project.

Date: _____

Pratik Mehata

B. Tech. CSE (8th Semester)

Enrollment No. [Enrollment Number]

University School of Information, Communication & Technology

Guru Gobind Singh Indraprastha University,

Sector-16C, Dwarka, New Delhi, India - 110078

Signature of the Guide

Prof. R.K. Purwar

GGSIPI University,

Dwarka, New Delhi, India

INDEX

S. No.	Topic	Page No.
1	Project Details	1
2	Project Objectives	1
3	Summary of Work Completed	1-3
4	Challenges Encountered & Resolved	8
5	Future Scope	9

1. PROJECT DETAILS

Project Title	RigCraft — AI-Powered PC Build Recommendation Engine
Student Name	Pratik Mehata
Enrollment Number	41216403222
Program	B.Tech. Computer Science & Engineering (8th Semester, Batch 2022-26)
Project Guide	(Prof. R.K. Purwar, USICT)
Reporting Period	Final Project Progress / Completion Report

2. PROJECT OBJECTIVE

The goal of this project is to build a web-based AI-powered PC build recommendation system for the Indian market. Instead of manually researching components across dozens of websites, users can simply describe their use case (gaming, video editing, programming etc.), set a budget in Indian Rupees, and instantly receive a complete, compatible PC build. I also wanted to ensure the backend engine remains completely modular and testable, with each recommendation step (intent classification, compatibility checking, budget optimization) implemented as an independent Python module, and the React frontend kept cleanly separated from the FastAPI backend.

3. SUMMARY OF WORK COMPLETED

Week 1: Project Planning & Architecture Design

- Defined the five-module backend pipeline architecture: IntentClassifier, ComponentDatabase, CompatibilityEngine, TierSelector, and BudgetOptimizer.
- Established the project folder structure separating the FastAPI backend (Python) from the React/Vite frontend (JavaScript).
- Designed the API contract: POST /api/recommend accepting query, budget, and resolution fields and returning a structured build JSON response.

Week 2: Component Dataset Curation

- Collected and cleaned real Indian market pricing data for 1,279 PC components across five categories: 304 CPUs, 359 GPUs, 121 Motherboards, 296 RAM modules, and 199 Storage devices.
- Structured data as CSV files with consistent columns (Name, Price, Socket, TDP, etc.) compatible with Pandas DataFrame loading.
- Implemented automatic performance tier classification (Entry / Mid / High / Pro) based on INR price thresholds for each component type.
-

Week 3: Intent Classifier Module

- Built the weighted keyword scoring intent classifier supporting nine use-case categories: FPS Gaming, Esports, AAA Gaming, General Gaming, CAD/3D Modeling, Video Editing, Streaming, Programming, and Office/Productivity.
- Designed the IntentRequirements dataclass mapping each intent to CPU class, GPU class, priority, minimum RAM, and resolution requirements.
- Validated classifier accuracy across 50 test inputs covering common user phrasings (e.g., 'valorant', 'blender', 'docker', 'study').

Week 4: Compatibility Engine Module

- Implemented CPU-to-motherboard socket compatibility checking using a SOCKET_FAMILIES registry grouping compatible socket types (LGA1700, AM4, AM5, LGA1200, Threadripper).
- Added RAM generation validation ensuring DDR4 RAM is only matched to DDR4 motherboards and DDR5 to DDR5 boards.
- Built the PSU wattage calculation model: $(\text{CPU TDP} + \text{GPU TDP} + 50\text{W overhead}) \times 1.35$ safety multiplier, rounded up to the nearest standard PSU tier.

Week 5: Budget Optimizer — Three Strategies

- Implemented the High-End strategy (above Rs. 2,00,000) maximizing absolute performance using top-30 CPU/GPU combinations sorted by composite performance score.
- Implemented the Mid-Range strategy (Rs. 50,000–Rs. 2,00,000) maximizing performance-per-rupee using a value scoring formula: $\text{perf_score} / (\text{total_cost} / 1000) \times (1 + \text{budget_usage} \times 0.5)$.

- Implemented the Budget strategy (below Rs. 50,000) finding the minimum viable compatible build and attempting upgrades when significant budget headroom remains.

Week 6: Intent-Aware Component Class Mapping

- Designed the `_INTENT_CLASS_MAP` table specifying the correct CPU and GPU performance class for each combination of intent and strategy level.
- Validated that a Programming build at Rs. 80,000 correctly selects a High-class CPU with Entry-class GPU, while a Gaming build at the same budget selects Mid CPU with High GPU.
- Integrated the class mapping into all three optimizer strategies, replacing generic price-range filtering.

Week 7: AI Fallback Module

- Integrated Google Gemini 1.5 Flash as the AI fallback provider using the google-genai SDK with a clean `_GeminiProvider` class abstraction.
- Designed a structured prompt instructing Gemini to return strictly formatted JSON with all required component fields.
- Implemented JSON response validation (required keys, positive prices, model name presence) with graceful fallback to hardcoded template builds if API fails.

Week 8: FastAPI Backend & REST API

- Built the production-grade FastAPI application with Pydantic v2 request/response validation, CORS middleware, and a global exception handler.
- Implemented environment-based configuration: `ENVIRONMENT=production` disables Swagger UI, raises logging level to `WARNING`, and hides internal error details from API responses.
- Registered the AI Panel router providing four endpoints: `/api/ai/explain`, `/api/ai/chat`, `/api/ai/upgrades`, and `/api/ai/compare`.

Week 9: React Frontend — Build Configuration UI

- Built the main `App.jsx` with intent selector cards (7 use-case categories with emoji icons, labels, and descriptions), budget slider (Rs. 10,000–Rs. 4,00,000 with INR presets), and resolution buttons (1080p / 1440p / 4K).
- Implemented the `GlassCard` component with glass-morphism styling (backdrop-filter blur, dark translucent background) used throughout the interface.
- Added the bouncing ball `Loader` component using CSS pseudo-element animations with Styled Components, displaying a blinking 'OPTIMIZING...' label during API calls.

Week 10: React Frontend — Build Result Display

- Built the ComponentRow component with expandable specification drawer, colored left-border accent per component type, and icon badges.
- Implemented the interactive SVG DonutChart displaying budget breakdown across components with hover state (segment highlight, center percentage display) and a right-side legend.
- Added the copy-to-clipboard feature exporting the complete build as formatted plain text with component names, prices, and PSU/cabinet recommendations.

Week 11: AI Panel — Four Feature Tabs

- Built the AIPanel component with four tabs: Explain (on-demand build explanation), Chat (multi-turn conversation with last 6 messages as context), Upgrades (current specs input form with bottleneck analysis), and Compare (dual-build side-by-side comparison).
- Implemented the MarkdownText renderer converting Gemini markdown responses (##, ###, bullets, bold) to styled JSX without external library dependencies.
- Integrated all four AI Panel tabs with their respective FastAPI endpoints, with loading spinners and error display for each tab independently.

Week 12: Deployment, Security & Bug Fixes

- Deployed the FastAPI backend to Render with Python version pinned to 3.12 (via .python-version file) to resolve Pandas 2.2.x C++ compilation incompatibility with Python 3.14.
- Deployed the React frontend to Vercel with VITE_API_BASE environment variable pointing to the Render backend URL.
- Resolved security issues including accidental API key exposure in git history using git filter-branch, added .env to .gitignore, and updated CORS ALLOWED_ORIGINS to the specific Vercel production URL.
- Fixed google.generativeai deprecation error by migrating to the google-genai SDK and updating all Gemini API calls to the new client.models.generate_content() interface.

4. CHALLENGES ENCOUNTERED & RESOLVED

Python 3.14 vs Pandas Compilation Incompatibility

Render's default Python environment was upgraded to Python 3.14 during deployment. Pandas 2.2.x uses C++ extensions compiled with Cython that failed to compile under Python 3.14 due to a breaking change in the handling of the `[[maybe_unused]]` attribute, producing a ninja build failure on step [122/151]. This was resolved by adding a `.python-version` file to the backend directory pinning the runtime to Python 3.12.0, which is fully compatible with all project dependencies.

Google Generative AI SDK Deprecation

The `google.generativeai` package was deprecated and removed from active support after `google-generativeai 0.8.x`. Attempts to use the new `google.genai` SDK with the import syntax `'from google import genai'` failed at runtime with the error `'cannot import name genai from google'`. The correct import for the new SDK is `'import google.generativeai as genai'` for the legacy package or using the `google-genai` package with `'from google import genai'` only after confirming the correct package version. The final resolution was maintaining `google-generativeai >= 0.8.0` with the `GenerativeModel('gemini-1.5-flash')` interface, which is stable and well-documented.

CORS Preflight (OPTIONS) Blocked in Production

After deploying the backend to Render and the frontend to Vercel, all API calls from the frontend returned network errors. Browser developer tools showed that preflight `OPTIONS` requests to `/api/recommend` were returning `HTTP 400`. The cause was that the FastAPI CORS middleware was configured with `allow_methods=['GET', 'POST']` without including `OPTIONS`. Browsers automatically send an `OPTIONS` preflight request before any cross-origin `POST`, and without `OPTIONS` being explicitly permitted, the middleware rejected them. This was resolved by adding `'OPTIONS'` to the `allow_methods` list and adding `'Authorization'` and `'Accept'` to `allow_headers`.

AI Fallback Not Loading Despite Valid API Key

The Gemini API key was correctly set in the `.env` file but the AI fallback reported `'No AI API key found'` on startup. The root cause was that `python-dotenv` was not installed in the virtual environment, so the `.env` file was never read and `os.getenv('GEMINI_API_KEY')` returned `None`. A secondary issue was that the `.env` file lacked a trailing newline, causing the last key-value pair (`ALLOWED_ORIGINS=*`) to be parsed incorrectly. Both issues were resolved by running `pip install python-dotenv` and appending a newline to the `.env` file.

Budget Optimizer Falling Back to AI for Mid-Range Budgets

The budget optimizer was incorrectly falling back to AI even for budgets of Rs. 1,55,000 and Rs. 2,95,000 where adequate database components existed. The root cause was a strict allocation floor in the mid-range strategy requiring CPU+GPU combined cost to be at least 40% of the budget. For intent categories like Programming with Entry-class GPU requirements, the CPU+GPU total was only ~24% of budget, failing the floor check. Additionally, the 4K resolution filter excluded all Entry-class GPUs since they target 1080p, leaving zero eligible GPUs. Both issues were resolved by removing the allocation floor (keeping only a ceiling), and implementing automatic resolution filter relaxation when no GPUs match the target resolution.

5. FUTURE SCOPE

As detailed in the dissertation, pending future enhancements include:

- **Real-Time Price Scraping:** Integrating automated scrapers targeting Amazon.in, Flipkart, and MDComputers to update component prices daily, ensuring recommendations always reflect current Indian market pricing rather than static dataset snapshots.
- **Persistent User Accounts and Build History:** Implementing a production-grade authentication backend (PostgreSQL or MongoDB) enabling users to save, name, and revisit previous builds, and tracking build trends over time.
- **Bottleneck Analyzer:** Adding a visual CPU-GPU balance chart showing the performance match percentage for the specified use case and resolution, with specific upgrade recommendations to resolve significant imbalances.
- **PDF Build Report Export:** Generating a one-click downloadable PDF report with component specifications, compatibility results, budget breakdown chart, and purchasing links for the Indian market.
- **Mobile Application:** Building a React Native mobile app consuming the existing FastAPI backend, with offline component database caching and push notifications for price drops on saved builds.